Singleton Pattern Implementation Guide for Login System

📋 Overview

This implementation uses the **Singleton Design Pattern** to maintain centralized login log entries across your ClothSphere system. The pattern ensures only one instance of the `LoginLogger` exists throughout the application lifecycle.

🎯 What is Singleton Pattern?

The Singleton pattern:

- ✅ Ensures a class has **only one instance**
- ✅ Provides a **global point of access** to that instance
- ✅ Is useful for managing **shared resources** (like logs, configuration, database connections)

📁 Implementation Steps

Step 1: Create the LoginLogger Class

File: `com.clothsphere.util.LoginLogger.java`

Key features:

- **Private constructor** - prevents direct instantiation
- **Static instance variable** - holds the single instance
- **Synchronized getInstance()** - ensures thread-safe instance creation
- **Thread-safe log storage** - uses synchronized list for concurrent access

```java
// Key Singleton implementation parts:
private static LoginLogger instance;  // Single instance
private LoginLogger() { ... }         // Private constructor
public static synchronized LoginLogger getInstance() { ... }
```

Step 2: Update Controllers

A. LoginController (HR Manager, Factory Manager, Sales Executive, Customer Officer, Employees)

File: com.clothsphere.controller.LoginController.java

Changes made:

java

```java
// Add at class level
private final LoginLogger logger = LoginLogger.getInstance();

// In processSystemUserLogin method:
logger.logSuccessfulLogin(username, userRole, "LoginController");
logger.logFailedLogin(username, "LoginController", "reason");
logger.logFirstTimeLogin(username, "LoginController");

// In logout method:
logger.logLogout(currentUser.getUserName(), currentUser.getRole());
```

B. ProfileController (Inventory Manager)

File: com.clothsphere.controller.IM.ProfileController.java

Changes made:

java

```java
// Add at class level
private final LoginLogger logger = LoginLogger.getInstance();

// Logging is primarily handled in LoginController
// But you can add custom events if needed
```

C. ProductOfficerAuthController (Product Officer)

File: com.clothsphere.controller.CPM.ProductOfficerAuthController.java

Changes made:

java

```
// Add at class level
private final LoginLogger logger = LoginLogger.getInstance();

// In dashboard method for failed access:
logger.logFailedLogin("unknown", "ProductOfficerAuthController", "Session expired");
```

🔧 How It Works

Flow Diagram

```
User Login Attempt
      ↓
LoginController.processSystemUserLogin()
      ↓
Validate Credentials
      ↓
Success? ──YES→ logger.logSuccessfulLogin()
  |           ↓
  |         Redirect to appropriate dashboard
  |           ↓
  |         (Inventory Manager → uses ProfileController)
  |         (Product Officer → uses ProductOfficerAuthController)
  |
  NO → logger.logFailedLogin()
       ↓
     Redirect to login page
```

Singleton Instance Usage

```
Application Startup
      ↓
First call to LoginLogger.getInstance()
      ↓
Creates single LoginLogger instance
      ↓
All subsequent calls return SAME instance
      ↓
All controllers share the SAME log storage
```

📊 Log Entry Types

1. Successful Login

```
java

logger.logSuccessfulLogin(username, role, controller);
```

When: User provides correct credentials **Status:** `SUCCESS`

2. Failed Login

```
java

logger.logFailedLogin(username, controller, reason);
```

When: Invalid credentials, user not found, missing password **Status:** `FAILED`

3. First-Time Login

```
java

logger.logFirstTimeLogin(username, controller);
```

When: Employee logs in for first time (logCount = 0) **Status:** `FIRST_LOGIN`

4. Logout

```
java

logger.logLogout(username, role);
```

When: User logs out **Status:** `LOGOUT`

🔍 Retrieving Logs

Get All Logs

```
java

LoginLogger logger = LoginLogger.getInstance();
List<LoginLogEntry> allLogs = logger.getAllLogs();
```

Get Logs by User

```java
List<LoginLogEntry> userLogs = logger.getLogsByUsername("john_doe");
```

Get Logs by Role

```java
List<LoginLogEntry> employeeLogs = logger.getLogsByRole("employee");
```

Get Logs by Status

```java
List<LoginLogEntry> failedLogins = logger.getLogsByStatus("FAILED");
```

Get User Statistics

```java
LoginLogger.LoginStatistics stats = logger.getUserStatistics("john_doe");
System.out.println("Successful logins: " + stats.getSuccessfulLogins());
System.out.println("Failed logins: " + stats.getFailedLogins());
System.out.println("Total logouts: " + stats.getLogouts());
```

🎨 Controller Responsibilities

Controller	Role	Responsibility
LoginController	All roles	Handles login authentication and logging
ProfileController	Inventory Manager	Profile management (accesses shared logger)
ProductOfficerAuthController	Product Officer	Dashboard access (accesses shared logger)

Important: All controllers access the **SAME singleton instance**, so all logs are centralized.

🧪 Testing the Singleton Pattern

Test 1: Verify Single Instance

java

```java
LoginLogger logger1 = LoginLogger.getInstance();
LoginLogger logger2 = LoginLogger.getInstance();

// Should be true - same instance
System.out.println(logger1 == logger2); // true
```

Test 2: Verify Shared Logs

java

```java
// Controller 1
LoginLogger logger1 = LoginLogger.getInstance();
logger1.logSuccessfulLogin("user1", "employee", "LoginController");

// Controller 2
LoginLogger logger2 = LoginLogger.getInstance();
System.out.println(logger2.getTotalLogCount()); // Should be 1

logger2.logSuccessfulLogin("user2", "hr-manager", "ProfileController");
System.out.println(logger1.getTotalLogCount()); // Should be 2
```

Test 3: Test Thread Safety

java

```java
// Multiple threads accessing same instance
for (int i = 0; i < 10; i++) {
    new Thread(() -> {
        LoginLogger logger = LoginLogger.getInstance();
        logger.logSuccessfulLogin("user" + Thread.currentThread().getId(),
                "employee", "TestController");
    }).start();
}
```

📈 Console Output Examples

Successful Login

```
=== LOGIN SUCCESS LOGGED ===
User: john_doe
Role: employee
Controller: LoginController
Time: 2025-10-14 10:30:45
Total logs: 1
```

Failed Login

```
=== LOGIN FAILURE LOGGED ===
User: jane_smith
Controller: LoginController
Reason: Incorrect password
Time: 2025-10-14 10:31:20
```

Logout

```
=== LOGOUT LOGGED ===
User: john_doe
Role: employee
Time: 2025-10-14 11:00:00
```

🔐 Security Considerations

1. **No Password Storage:** Logs never store passwords, only usernames and results
2. **Thread-Safe:** Uses synchronized collections for concurrent access
3. **Read-Only Access:** Public methods return unmodifiable lists
4. **Role-Based Access:** Only admins should view full logs

📊 Optional: Admin Dashboard for Logs

Add this endpoint to **LoginController** to view all logs:

java

```java
@GetMapping("/admin/loginLogs")
public String viewLoginLogs(HttpSession session, Model model) {
    SystemUser currentUser = (SystemUser) session.getAttribute("currentUser");

    // Only Factory Manager and HR Manager can view
    if (currentUser == null ||
        (!currentUser.getRole().equals("Factory Manager") &&
         !currentUser.getRole().equals("HR Manager"))) {
        return "redirect:/dashboard";
    }

    LoginLogger logger = LoginLogger.getInstance();
    model.addAttribute("allLogs", logger.getAllLogs());
    model.addAttribute("totalLogs", logger.getTotalLogCount());
    model.addAttribute("successLogs", logger.getLogsByStatus("SUCCESS").size());
    model.addAttribute("failedLogs", logger.getLogsByStatus("FAILED").size());

    return "loginLogsView";
}
```

✅ Benefits of This Implementation

1. **Centralized Logging:** All login activities in one place
2. **Consistent:** Same logging mechanism across all controllers
3. **Memory Efficient:** Only one logger instance exists
4. **Thread-Safe:** Can handle concurrent logins
5. **Easy to Query:** Simple methods to retrieve specific logs
6. **Audit Trail:** Complete history of login activities
7. **Debugging:** Easy to track authentication issues

🚀 Next Steps

1. ✅ Copy `LoginLogger.java` to `com.clothsphere.util` package
2. ✅ Update `LoginController.java` with logging calls

3. ✅ Update `ProfileController.java` (if needed)
4. ✅ Update `ProductOfficerAuthController.java` (if needed)
5. 🔄 Test login flows for each role
6. 🔄 Verify logs are being created
7. 🔄 (Optional) Create admin view to display logs

📚 Assignment Submission Points

For your university lecture, highlight:

1. **Singleton Pattern Implementation:**
   - Private constructor
   - Static instance variable
   - Synchronized getInstance() method
   - Thread-safe log storage
2. **Use Case:** Centralized login log management
3. **Benefits:**
   - Single point of access
   - Consistent logging across controllers
   - Memory efficient
   - Thread-safe
4. **Controllers Using Singleton:**
   - LoginController (main)
   - ProfileController (Inventory Manager)
   - ProductOfficerAuthController (Product Officer)
5. **Log Entry Types:** SUCCESS, FAILED, FIRST_LOGIN, LOGOUT

💡 Key Takeaway

> "The Singleton pattern ensures only one instance of LoginLogger exists throughout the application, providing centralized and consistent login activity tracking across all controllers handling different user

> roles."

Troubleshooting

Issue: Logs not appearing

Solution: Ensure you're calling `LoginLogger.getInstance()` not creating new instances

Issue: NullPointerException

Solution: Check that logger instance is initialized at class level

Issue: Logs showing in one controller but not another

Solution: Verify both controllers use `getInstance()` - they should share the same logs

---

End of Implementation Guide